المملكة المغربية
ⵜⴰⴳⵍⴷⵉⵜ ⵏ ⵍⵎⵖⵔⵉⴱ
وزارة التربية الوطنية
والتعليم الأولي والرياضة
ⵜⴰⵎⴰⵡⴰⵙⵜ ⵏ ⵓⵙⴳⵎⵉ ⴰⵏⴰⵎⵓⵔ
ⴷ ⴰⵙⵙⵍⵎⴷ ⴰⵎⵣⵡⴰⵔⵓ ⴷ ⵜⵓⵏⵏⵓⵏⵜ

Examen National du Brevet de Technicien Supérieur (BTS) 1

Session 2026

- Correction -

Filière :	**Développement des Applications Informatiques -DAI-**	**Durée :** 2 heures
Épreuve :	**Développement Mobile Android**	**Coefficient :** 45

DOSSIER I : STRUCTURE & UI

1. Architecture Android

1. Les composants principaux :
   - **Activity :** C'est l'écran de l'application
   - **Services :** Un Service = code qui tourne en arrière-plan : **musique qui continue même si tu quittes l'app, téléchargement de données, GPS en arrière-plan..** Un Service permet d'exécuter une tâche sans interface utilisateur.
   - **Broadcast Receivers :** C'est un composant qui écoute des événements du système : **Batterie faible, Téléphone redémarré, Mode avion..** Android envoie des messages (broadcast), le receiver les écoute.
   - **Content Providers :** Permet de partager des données entre applications, comme : **Contacts du téléphone, Galerie..** Une app peut lire ces données via ContentProvider
2. Rôles :
   - **Activity** : représente une interface utilisateur (écran).
   - **Layout XML** : définit la structure graphique de l'interface.
   - **AndroidManifest.xml** : déclare les composants et permissions de l'application.

2. Interface utilisateur

1. Code XML :

```
<LinearLayout
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <EditText
        android:id="@+id/nomPatient"
        android:hint="Nom␣du␣patient"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/btnAjouter"
        android:text="Ajouter"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
```

```
    <ListView
        android:id="@+id/listePatients"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

2. Types de vues :
   — EditText
   — Button
   — ListView

3. Layouts Android

1. Différence :
   — **LinearLayout** : organise les éléments verticalement ou horizontalement.
   — **ConstraintLayout** : permet un positionnement flexible basé sur des contraintes.
2. Exemple :

```
<LinearLayout
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <Button android:text="OK"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    <Button android:text="Annuler"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

4. Gestion des événements

1. Code Java :

```
Button btn = findViewById(R.id.btnAjouter);
EditText nom = findViewById(R.id.nomPatient);

btn.setOnClickListener(v -> {
    String n = nom.getText().toString();
```

2. Toast :

```
    Toast.makeText(this, n, Toast.LENGTH_SHORT).show();
});
```

DOSSIER II : CYCLE DE VIE & STOCKAGE

Classes

```
class Patient {
    int id;
    String nom;
    int age;
}

class RendezVous {
    String date;
    String description;
}
```

1. Cycle de vie

1. Méthodes :
   — onCreate()
   — onStart()
   — onResume()
   — onPause()
   — onStop()
   — onDestroy()
2. Rôles :
   — onCreate() : initialisation de l'activité.
   — onPause() : sauvegarde temporaire des données.
   — onDestroy() : nettoyage des ressources.
3. Exemple :
   — onCreate : charger la liste des patients.
   — onPause : sauvegarder les données.
   — onDestroy : fermer la base de données.

2. SharedPreferences

1. Sauvegarde :

```
SharedPreferences sp = getSharedPreferences("app", MODE_PRIVATE);
sp.edit().putString("nom", "Ali").apply();
```

2. Lecture :

```
String nom = sp.getString("nom", "");
```

3. Première ouverture :
   — Principe : stocker un booléen "firstLaunch"

```
boolean first = sp.getBoolean("firstLaunch", true);

if(first){
    // afficher splash
    startActivity(new Intent(this, WelcomeActivity.class));
    sp.edit().putBoolean("firstLaunch", false).apply();
}
```

3. Fichiers

1. Différence :
   — Interne : privé à l'application.
   — Externe : accessible par l'utilisateur.
2. Sauvegarde :

```
FileOutputStream fos = openFileOutput("data.txt", MODE_APPEND);
String id = UUID.randomUUID().toString();

BufferedWriter writer = new BufferedWriter(
        new OutputStreamWriter(openFileOutput("patients.txt",
           MODE_APPEND)));

writer.write(id + ";" + "Amina" + ";" + 28);
writer.newLine();
writer.close();
fos.close();
```

   Lecture :

```
ArrayList<Patient> liste_patients = new ArrayList<>();
BufferedReader reader = new BufferedReader(
        new InputStreamReader(openFileInput("patients.txt")));

String ligne;
while ((ligne = reader.readLine()) != null) {
    String[] p = ligne.split(";");
    Patient patient = new Patient(
        Integer.parseInt(p[0]),
        p[1],
        Integer.parseInt(p[2])
    );
    liste_patients.add(patient)
}
reader.close();
```

3. Avantage / Inconvénient :
   — Avantage : simple et rapide
   — Inconvénient : difficile à gérer et non structuré

DOSSIER III : LISTES & SQLITE

1. AdapterView

1. Rôles :
   — Adapter : relie données et vue
   — ListView : affiche les données
2. Exemple :

```
ArrayAdapter<String> adapter =
new ArrayAdapter<>(this,
android.R.layout.simple_list_item_1, liste);

listView.setAdapter(adapter);
```

2. SQLite

1. Création :

```
public class DBHelper extends SQLiteOpenHelper {

    public DBHelper(Context c) {
        super(c, "db", null, 1);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE␣TABLE␣Patient(id␣INTEGER␣PRIMARY␣KEY,␣nom␣
           TEXT,␣age␣INTEGER)");
        db.execSQL("CREATE␣TABLE␣RendezVous␣(" +
            "id␣INTEGER␣PRIMARY␣KEY␣AUTOINCREMENT,␣" +
            "date␣TEXT,␣description␣TEXT,␣patient_id␣INTEGER,␣" +
            "FOREIGN␣KEY(patient_id)␣REFERENCES␣Patient(id))");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int
       newVersion) {
        db.execSQL("DROP␣TABLE␣IF␣EXISTS␣Patient");
        onCreate(db);
    }
}
```

CRUD DAO :

```
public void insertPatient(String nom, int age) {
    SQLiteDatabase db = this.getWritableDatabase();

    ContentValues values = new ContentValues();
    values.put("nom", nom);
    values.put("age", age);

    db.insert("Patient", null, values);
}
//////////////////////////////
public void updatePatient(int id, String nom, int age) {
    SQLiteDatabase db = this.getWritableDatabase();

    ContentValues values = new ContentValues();
    values.put("nom", nom);
    values.put("age", age);

    db.update("Patient", values, "id=?", new String[]{String.valueOf(
       id)});
}
//////////////////////////////
public void deletePatient(int id) {
    SQLiteDatabase db = this.getWritableDatabase();

    db.delete("Patient", "id=?", new String[]{String.valueOf(id)});
}
//////////////////////////////
```

```java
public ArrayList<Patient> getAllPatients() {

    ArrayList<Patient> liste = new ArrayList<>();
    SQLiteDatabase db = this.getReadableDatabase();

    Cursor cursor = db.rawQuery("SELECT␣*␣FROM␣Patient", null);

    if (cursor.moveToFirst()) {
        do {
            int id = cursor.getInt(0);
            String nom = cursor.getString(1);
            int age = cursor.getInt(2);

            Patient p = new Patient(id, nom, age);
            liste.add(p);

        } while (cursor.moveToNext());
    }

    cursor.close();

    return liste;
}
//////////////////////////////
public void insertRendezVous(RendezVous rdv) {
    SQLiteDatabase db = dbHelper.getWritableDatabase();

    ContentValues values = new ContentValues();
    values.put("date", rdv.date);
    values.put("description", rdv.description);
    values.put("patient_id", rdv.patientId);

    db.insert("RendezVous", null, values);
}
//////////////////////////////
public ArrayList<RendezVous> getRendezVousByPatient(int patientId) {

    ArrayList<RendezVous> liste = new ArrayList<>();


    Cursor cursor = db.rawQuery(
            "SELECT␣*␣FROM␣RendezVous␣WHERE␣patient_id=?",
            new String[]{String.valueOf(patientId)}
    );

    if (cursor.moveToFirst()) {
        do {
            RendezVous rdv = new RendezVous(
                    cursor.getInt(0),
                    cursor.getString(1),
                    cursor.getString(2),
                    cursor.getInt(3)
            );
            liste.add(rdv);
        } while (cursor.moveToNext());
```

```
    }

    cursor.close();
    return liste;
}
```

3. Exceptions

1. Cas :
   — NumberFormatException
   — Champ vide
2. Code :

```
try {
    int age = Integer.parseInt(edit.getText().toString());
} catch(NumberFormatException e){
    Toast.makeText(this,"Erreur",Toast.LENGTH_SHORT).show();
}

if(edit.getText().toString().isEmpty()){
    edit.setError("Champ␣obligatoire");
}
```

DOSSIER IV : FIREBASE

1. Firebase

```
DatabaseReference ref = FirebaseDatabase.getInstance()
.getReference("patients");

ref.push().setValue(patient);

ref.addValueEventListener(new ValueEventListener(){
    public void onDataChange(DataSnapshot ds){
        for(DataSnapshot d : ds.getChildren()){
            Patient p = d.getValue(Patient.class);
        }
    }
});
```

2. MVVM

1. Différence :
   — MVC : logique mélangée
   — MVVM : séparation claire
2. Rôles :
   — ViewModel : contient les données
   — LiveData : observe les changements

3. Architecture globale

- Activity (UI)
- ViewModel (logique)
- Repository (accès données)
- SQLite + Firebase
- SharedPreferences